

Mail, typed by hand

One Postfix and Dovecot server, two client machines, and a message pushed through both protocols a line at a time — SMTP to hand it over, IMAP to read it back, and the server's own log in between. From a running lab.

Command `qlab run mail-lab`

VMs `mail-lab-server` · `mail-lab-client1` (alice) · `mail-lab-client2` (bob)

LAN `192.168.100.0/24`, isolated between the three VMs

Credentials `labuser` / `labpass` · mail users `alice`, `bob` / `labpass`

Outcome `qlab test mail-lab` → 6 exercises, 39 checks, all passed

1. Three machines, two protocols

```
mail-lab-server      192.168.100.1/24
mail-lab-client1     192.168.100.2/24
mail-lab-client2     192.168.100.3/24
```

```
mail-lab-server
alice:1001:/home/alice
bob:1002:/home/bob
```

One server and two clients, one per user. The split is deliberate: with alice and bob on separate machines, every message has to cross a wire, and both halves of the job — handing a message over, and fetching it back — are visible as network traffic rather than as files appearing in a directory.

alice and bob are ordinary Unix accounts on the server, with ordinary home directories. That is the whole user database. Postfix and Dovecot both read it, which is why this lab needs no LDAP, no SQL and no virtual-mailbox tables.

2. What the server is running

```
$ the two daemons, and the ports they hold
active active
0.0.0.0:993 "dovecot",pid=6303,fd=34
0.0.0.0:25 "master",pid=6200,fd=13
0.0.0.0:143 "dovecot",pid=6303,fd=33
[::]:993 "dovecot",pid=6303,fd=35

$ postconf -n      (only what differs from the defaults)
home_mailbox = Maildir/
inet_interfaces = all
inet_protocols = ipv4
mailbox_size_limit = 0
mydestination = $myhostname, mail.lab, localhost.localdomain, localhost
mydomain = mail.lab
myhostname = mail.lab
mynetworks = 127.0.0.0/8, 192.168.100.0/24
myorigin = $mydomain
smtpd_banner = $myhostname ESMTP mail.lab
smtpd_relay_restrictions = permit_mynetworks, reject_unauth_destination

$ where Dovecot expects to find the mail
mail_location = maildir:~/Maildir
protocols = imap
auth_mechanisms = plain login
disable_plaintext_auth = no
```

Two daemons, four ports, and one clear division of labour: **Postfix owns port 25** and moves mail between machines; **Dovecot owns 143 and 993** and hands mail to people. They never speak to each other. Their only shared object is the Maildir under each user's home — one writes it, the other reads it.

`postconf -n` prints only the settings that differ from the compiled-in defaults, which is the fastest way to see what a mail server has actually been told to do. `myhostname` and `mydomain` give it a name; `mydestination` lists the domains it considers its own; `home_mailbox = Maildir/` chooses the storage format; `mynetworks` decides who may post through it — section 7 comes back to that one.

On the Dovecot side, `mail_location` points at the same Maildir, and `disable_plaintext_auth = no` is a lab convenience: it lets you log in over plain 143 with `telnet` and watch, which is exactly what section 6 does.

3. SMTP, one line at a time

```
$ telnet 192.168.100.1 25      - the whole submission, typed by hand
Trying 192.168.100.1...
Connected to 192.168.100.1.
Escape character is '^J'.
220 mail.lab ESMTP mail.lab
250-mail.lab
250-PIPELINING
250-SIZE 10240000
250-VRFY
250-ETRN
250-ENHANCEDSTATUSCODES
250-8BITMIME
250-DSN
250-SMTPUTF8
250 CHUNKING
250 2.1.0 Ok
250 2.1.5 Ok
354 End data with <CR><LF>.<CR><LF>
250 2.0.0 Ok: queued as E517414B0
221 2.0.0 Bye
Connection closed by foreign host.
```

This is a complete mail submission with nothing between the fingers and the socket. Every line the client sends is a short command; every line the server sends back begins with a three-digit code.

The shape is always the same. `EHL0` announces the client and asks what the server supports — the `250-` lines are that answer, a capability list ending with a `250` (space, not dash) on the last one. `MAIL FROM` opens an envelope. `RCPT TO` puts a recipient on it, once per recipient. `DATA` switches from commands to content, and the content ends with a lone dot on its own line, which is why `354` spells out the terminator. Then the server takes responsibility: `250 Ok: queued as ...` means the message is on disk and the sender may forget about it.

The envelope and the headers are different things. `MAIL FROM` / `RCPT TO` are the envelope, which is what actually routes the message; `From:`, `To:` and `Subject:` are typed inside `DATA` and are just text. Nothing checks that they agree — that gap is where forged sender addresses live, and why SPF, DKIM and DMARC had to be invented on top.

4. The same message, from the server's side

```
$ the life of that one message, on the server
queue id: E517414B0

Sep 14 14:02:54 postfix/smtpd[6927]: E517414B0: client=unknown[192.168.100.2]
Sep 14 14:02:56 postfix/cleanup[6930]: E517414B0: message-id=<>
Sep 14 14:02:56 postfix/qmgr[6203]: E517414B0: from=<alice@mail.lab>, size=260, nrcpt=1 (queue active)
Sep 14 14:02:56 postfix/local[6931]: E517414B0: to=<bob@mail.lab>, relay=local, delay=3, delays=3/0/0/0,
dsn=2.0.0, status=sent (delivered to maildir)
Sep 14 14:02:56 postfix/qmgr[6203]: E517414B0: removed
```

That queue ID is the thread to pull. Every line Postfix logs about a message carries it, so grepping one ID gives the message's whole life in order.

Five lines, five processes, and each one is a separate program: `smtpd` took the connection, `cleanup` normalised the headers and gave it a message-id, `qmgr` put it in the active queue, `local` delivered it, and `qmgr` removed it. Postfix is built this way on purpose — small programs, one queue, minimal privilege — and the log is a direct reflection of that architecture.

The line worth memorising is the delivery one. `to=`, `relay=`, `delay=`, `dsn=`, `status=` — when mail goes missing, this is the line that says whether your server ever had it, what it did with it, and who it blamed.

5. Maildir: one file per message

```
$ where it actually landed
/home/bob/Maildir
/home/bob/Maildir/cur
/home/bob/Maildir/new
/home/bob/Maildir/tmp

new 1 file(s)
cur 2 file(s)

14:02:56.9645628880 337 1789394576.Vfe01I14b7M967374.mail-lab-server

$ the message as stored – the headers Postfix added are at the top
Return-Path: <alice@mail.lab>
X-Original-To: bob@mail.lab
Delivered-To: bob@mail.lab
Received: from client1.lab (unknown [192.168.100.2])
    by mail.lab (Postfix) with ESMTP id E517414B0
    for <bob@mail.lab>; Mon, 14 Sep 2026 14:02:53 +0000 (UTC)
Subject: sent by hand
From: alice@mail.lab
To: bob@mail.lab

Typed straight into the socket.
```

`status=sent (delivered to maildir)` meant this. Three directories, one file per message, no locking and no shared index — that is the entire Maildir format, and it is why it survives crashes and NFS in a way that a single `mbx` file does not.

The subdirectories carry the state. A new delivery is written into `tmp` and then moved into `new`; when a client opens the mailbox, the message moves to `cur` and its flags get appended to the filename. So the counts above are not decoration: `new` holds what has never been looked at, `cur` holds what has. Bob's earlier messages are already in `cur` because section 6's IMAP session opened his INBOX.

At the top of the stored file are the headers Postfix added on the way past. `Received:` is the important one — it records which host handed the message over, which host took it, and when. A real message accumulates one per hop, and the stack is read bottom-up.

6. IMAP, also one line at a time

```
$ telnet 192.168.100.1 143      - bob reads it, also by hand
Trying 192.168.100.1...
Connected to 192.168.100.1.
Escape character is '^]'.
* OK Waiting for authentication process to respond..
* OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE LITERAL+ STARTTLS AUTH=PLAIN
AUTH=LOGIN] Dovecot (Ubuntu) ready.
a OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE SORT SORT=DISPLAY THREAD=REFERENCES
THREAD=REFS THREAD=ORDEREDSUBJECT MULTIAPPEND URL-PARTIAL CATENATE UNSELECT CHILDREN NAMESPACE UIDPLUS
LIST-EXTENDED I18NLEVEL=1 CONDSTORE QRESYNC ESEARCH ESORT SEARCHRES WITHIN CONTEXT=SEARCH LIST-STATUS
BINARY MOVE SNIPPET=FUZZY PREVIEW=FUZZY PREVIEW STATUS=SIZE SAVEDATE LITERAL+ NOTIFY SPECIAL-USE] Logged
in
* LIST (\HasNoChildren) "." INBOX
b OK List completed (0.002 + 0.000 + 0.001 secs).
* FLAGS (\Answered \Flagged \Deleted \Seen \Draft)
* OK [PERMANENTFLAGS (\Answered \Flagged \Deleted \Seen \Draft *)] Flags permitted.
* 3 EXISTS
* 1 RECENT
* OK [UNSEEN 1] First unseen.
* OK [UIDVALIDITY 1789394123] UIDs valid
* OK [UIDNEXT 5] Predicted next UID
c OK [READ-WRITE] Select completed (0.016 + 0.000 + 0.015 secs).
* 1 FETCH (FLAGS () BODY[HEADER.FIELDS (FROM SUBJECT)] {47}
Subject: sent by hand
From: alice@mail.lab

)
* 2 FETCH (FLAGS () BODY[HEADER.FIELDS (FROM SUBJECT)] {47}
Subject: sent by hand
From: alice@mail.lab

)
* 3 FETCH (FLAGS (\Recent) BODY[HEADER.FIELDS (FROM SUBJECT)] {47}
Subject: sent by hand
From: alice@mail.lab

)
d OK Fetch completed (0.004 + 0.000 + 0.003 secs).
* BYE Logging out
e OK Logout completed (0.001 + 0.000 secs).
Connection closed by foreign host.
```

IMAP looks different from SMTP because it is a different shape of problem: not "take this message" but "let me work with a mailbox that stays on your disk". Each client command gets a tag (`a`, `b`, `c` ...) and the server's final reply to that command repeats it, so several

commands can be in flight at once. Lines beginning with `*` are untagged data the server volunteers along the way.

`LOGIN` authenticates and is answered with a capability list. `LIST` enumerates mailboxes. `SELECT INBOX` opens one and immediately reports what is in it — `EXISTS`, `RECENT`, the flags the mailbox supports, and `UIDVALIDITY`, the number a client checks to decide whether its cached copy is still trustworthy. `FETCH` then asks for specific parts of specific messages: here just two header fields, which is exactly what a mail client needs to draw an index without downloading anything.

`BODY.PEEK[...]` rather than `BODY[...]` is not a detail. Plain `BODY[]` sets the `\Seen` flag as a side effect of reading; `PEEK` does not. Every mail client you have used makes that choice on your behalf.

7. Who is allowed to post through this server

```
$ who is allowed to post through this server
mynetworks = 127.0.0.0/8, 192.168.100.0/24
smtpd_relay_restrictions = permit_mynetworks, reject_unauth_destination
smtpd_recipient_restrictions =

$ two recipients offered from a LAN client, one local and one not
220 mail.lab ESMTP mail.lab
250 CHUNKING
250 2.1.0 Ok
550 5.1.1 <ghost@mail.lab>: Recipient address rejected: User unknown in local recipient table
250 2.1.5 Ok
354 End data with <CR><LF>.<CR><LF>
250 2.0.0 Ok: queued as 91DF1AF2A
221 2.0.0 Bye

$ and what became of the one that was accepted for off-site delivery
Sep 14 14:05:03 postfix/smtpd[6927]: 91DF1AF2A: client=unknown[192.168.100.2]
Sep 14 14:05:05 postfix/cleanup[6930]: 91DF1AF2A: message-id=<>
Sep 14 14:05:05 postfix/qmgr[6203]: 91DF1AF2A: from=<alice@mail.lab>, size=213, nrcpt=1 (queue active)
Sep 14 14:05:05 postfix/smtp[7028]: 91DF1AF2A: to=<nobody@example.org>, relay=none, delay=4.1,
delays=4.1/0/0/0, dsn=5.1.0, status=bounced (Domain example.org does not accept mail (nullMX))
Sep 14 14:05:05 postfix/bounce[7029]: 91DF1AF2A: sender non-delivery notification: 9E2EAAFEE
Sep 14 14:05:05 postfix/qmgr[6203]: 91DF1AF2A: removed

messages in alice INBOX now: 1
hdr.from: MAILER-DAEMON@mail.lab (Mail Delivery System)
hdr.subject: Undelivered Mail Returned to Sender
```

Two recipients offered in one session, two different answers, and the difference is the single most important setting on a mail server.

`ghost@mail.lab` is refused with `550 ... User unknown in local recipient table`: the domain is one of `mydestination`, so the server is authoritative for it and knows there is no such user.

`nobody@example.org` is *accepted* — because the client is inside `mynetworks`, and `smtpd_relay_restrictions` begins with `permit_mynetworks`. The server agreed to carry mail to a domain that is not its own. That is relaying, and it is a normal, wanted thing for machines you trust.

`reject_unauth_destination` is what stops it being an **open relay**: it refuses that same recipient from anyone outside `mynetworks`. Getting this pair wrong is the classic way a new mail server ends up on every blocklist within a day.

Then the message that was accepted goes nowhere, and the log says why: no route to the destination, `status=bounced`, and Postfix generates a non-delivery notification back to the sender. Alice's INBOX now holds one message, from `MAILER-DAEMON`. Accepting a message and delivering it are two separate promises, and a bounce is the server keeping the second one it could not keep.

8. The client does exactly this, with a UI

The human client does exactly the two things done by hand above:

```
--- alice (client1) ~/.muttrc ---
set from = "alice@mail.lab"
set folder = "imap://alice:labpass@mail.lab:143/"
set spoolfile = "imap://alice:labpass@mail.lab:143/INBOX"
set smtp_url = "smtp://alice@mail.lab:25/"
--- bob (client2) ~/.muttrc ---
set from = "bob@mail.lab"
set folder = "imap://bob:labpass@mail.lab:143/"
set spoolfile = "imap://bob:labpass@mail.lab:143/INBOX"
set smtp_url = "smtp://bob@mail.lab:25/"
```

\$ and the same mailbox, asked of Dovecot from the server side
INBOX

messages in bob INBOX: 3

Four lines of `mutt` configuration, and they map one-to-one onto the two sessions above: `folder` and `spoolfile` are the IMAP side, `smtp_url` is the SMTP side. A mail client is two protocol clients in a trench coat — one to send, one to read — and they do not even have to point at the same server.

`doveadm` is the view from behind: the same mailbox, the same messages, read straight off the disk with no network in between.

9. Try it yourself

```
qlab run mail-lab
qlab shell mail-lab-client1      # alice
qlab shell mail-lab-client2      # bob
```

The intended path is the human one — `sudo -u alice bash`, then `mutt` — but the instructive path is the one above:

```
telnet 192.168.100.1 25      # EHLO / MAIL FROM / RCPT TO / DATA / .
telnet 192.168.100.1 143    # a LOGIN bob labpass / b SELECT INBOX / c FETCH 1 ...
```

Three things worth doing:

- Run `sudo tail -f /var/log/mail.log` on the server in one window while you type an SMTP session in another. Every command you type appears there.
- Send a message with a `From:` header that does not match `MAIL FROM`, then read it in mutt and see which one it shows you.
- Watch a file move from `new/` to `cur/` the moment you open the mailbox.

```
Exercise 1 – Mail Server Anatomy
  Exercise 1: All 12 checks passed
Exercise 2 – Send and Receive Mail
  Exercise 2: All 4 checks passed
Exercise 3 – SMTP Protocol
  Exercise 3: All 5 checks passed
Exercise 4 – IMAP Protocol
  Exercise 4: All 5 checks passed
Exercise 5 – Postfix Configuration
  Exercise 5: All 9 checks passed
Exercise 6 – Add a New User
  Exercise 6: All 4 checks passed
All 6 exercise(s) passed!
```

`qlab test mail-lab` runs 39 checks across the three VMs — including a full SMTP conversation, an IMAP login, and creating a third user from scratch to prove that "add a mail user" really does mean `adduser`.